

SokoPay customer app: security assessment

Date: 10 October 2026 · **Scope:** the customer Android app and every server endpoint it can reach (wallet, payments, transfers, cross-border, merchants, agents' cash-out, marketplace, KYC, notifications, statements, USSD, partner callbacks) · **Result:** 9 issues found, **all 9 fixed and re-tested**; 9 recommendations remain before real-customer production.

1. Summary

#	Severity	Issue	Status
1	Critical	Most wallet debits needed no PIN : sending to a SokoPay user, transfers to MoMo/bank, paying a merchant, bills/airtime/data from the wallet, insurance premiums. Anyone holding an open session (a stolen or unlocked phone, a copied token) could empty the wallet.	Fixed: the server now requires and checks the PIN on every wallet debit; the app asks for it.
2	High	Double payment on retry: wallet-to-wallet sends had no idempotency key, and the app made a new key on every tap for merchant pay, bills, data and transfers. A double tap or a retry after a dropped connection could pay twice.	Fixed: server-side idempotency on P2P; the app keeps one key per payment attempt.
3	Medium	Limits could be beaten by racing: balance and limit checks (daily/monthly caps, the GH¢ 500 cap after a PIN reset) ran before the ledger lock. Proven: 6 simultaneous GH¢ 40 sends moved GH¢ 240 against a GH¢ 100 cap.	Fixed: one debit at a time per customer (PostgreSQL advisory lock). The same test now stops at GH¢ 80.
4	Medium	A stolen refresh token wasn't treated as theft: reusing a spent one was refused, but the thief's or owner's other session continued, and two simultaneous refreshes could both succeed.	Fixed: reuse (or a race) now signs the person out everywhere.
5	Medium	Login revealed who has an account ("No account for that phone" vs "Wrong PIN", and a timing difference).	Fixed: one message and the same hashing time either way.
6	Medium	Number harvesting: recipient lookups allowed 120 per minute, enough to test which phone numbers use SokoPay.	Fixed: 60 lookups per hour per person.
7	Low	The API description (Swagger, schema) was public on servers.	Fixed: development only.
8	Low	Endpoints were public unless they said otherwise (framework default). No endpoint was exposed, but a future one could be.	Fixed: sign-in required by default; public ones opt out explicitly.
9	Low	On servers the API also accepted browser session cookies (portal logins).	Fixed: the API accepts app tokens only.

2. What was tested and passed

Attack-surface sweep (every one of the 90 API routes × every HTTP method): - **No anonymous access:** no route answers an anonymous caller except the deliberately public ones (sign-in, OTP, and partner callbacks, which are signature-checked). - **Forged tokens are rejected:** wrong key, alg=none, expired, signed-out, refresh used as access, a missing ID, and garbage. - **Logout is real:** signing out kills the session at once. - **Junk input never crashes anything:** negative or huge amounts, NaN, 1e309, 5,000-character fields, SQL and path-traversal strings, objects where text is expected. - **Roles are enforced:** a customer token is refused on agent and merchant routes.

Sign-in and accounts: - **OTP brute force:** a code is dead after 5 wrong tries, and only 5 codes can be requested an hour. - **PIN brute force:** 5 wrong PINs lock the account, and in-app PIN prompts count towards the same lock. - **PIN**

reset: a verified customer must also give their Ghana Card. - **After a reset:** sends are capped for 24 hours. - **Disabled or closed accounts:** they lose access immediately.

Money integrity: - **Amounts:** zero, negative, three-decimal, scientific notation, NaN and Infinity are all refused, and nothing moves. - **Basic rules:** no overdraft, no sending to yourself, held wallets can't send, and tier limits are enforced. - **Ledger:** amounts are whole pesewas only, every entry balances to zero, and row locks stop any wallet going negative (PostgreSQL concurrency tests).

Partner callbacks: - **Forged callbacks** (bad signature) are rejected, and nothing is credited. - **The callback body isn't trusted:** the server re-asks the partner for the status. - **A replayed callback credits once.** - **Unsigned remittance callbacks are refused.** - **USSD:** needs the gateway secret (compared in constant time), and every USSD money action asks for the PIN.

Other people's data: - **Payments, saved recipients, notifications:** another customer's can't be read or changed. - **Cash-out requests:** another customer's can't be approved. - **Push devices:** another customer's can't be removed. - **Activity and ID documents:** customers see only their own.

Leakage: - **Recipient names:** lookups show a short name ("Ama S."), never the full name or number. - **Errors:** they contain no internal details.

Android app (all three apps): - **Backups and traffic:** app data is excluded from cloud backup, plain HTTP is blocked except to a developer's own machine, and only the launcher screen is exposed to other apps. - **Secrets on the phone:** tokens are kept in Android encrypted storage, and nothing logs tokens, PINs or codes. - **Notification links:** payloads are validated, so a notification can't open an arbitrary screen. - **App lock:** after 2 minutes in the background.

Static and configuration checks: - **Django production check:** passes. The only warnings are the two HSTS options deliberately relaxed on the test server; production turns them on. - **Code checks:** the `bandit` security scan has no medium or high findings, and `ruff` is clean. - **Dependencies:** `pip-audit` finds no known-vulnerable packages.

3. Recommendations before real customers

These aren't defects in the pilot, but they matter for production:

1. **Release signing:** sign the apps with a real upload key (Play App Signing) instead of the debug key, and build with `--obfuscate --split-debug-info`.
 2. **Certificate pinning** in the app, so a malicious certificate authority or a compromised device can't intercept traffic.
 3. **Device integrity:** use the Google Play Integrity API (rooted or emulator detection), and limit risky actions on failing devices.
 4. **Screenshots and screen recording (FLAG_SECURE)** in the customer app. This is a product decision: it also blocks customers screenshotting receipts.
 5. **Self-hosted portal scripts:** serve the portal's CSS and JavaScript from your own server (not public CDNs), and add a Content-Security-Policy.
 6. **Separate token key:** a dedicated JWT signing key, rotated on a schedule, rather than reusing Django's `SECRET_KEY`.
 7. **Edge protection:** rate limits and a WAF at the reverse proxy (per-IP limits on sign-in and OTP routes).
 8. **Alerting:** alerts on spikes in PIN lockouts, OTP requests and refresh-token reuse; the events are already logged.
 9. **Independent penetration test** by an accredited firm before launch. The Bank of Ghana expects one; this assessment doesn't replace it.
-

4. How to re-run

```
cd backend
.venv/Scripts/python -m pytest apps/securitytests          # the attack suite (SQLite)
DATABASE_URL=postgres:///... .venv/Scripts/python -m pytest apps/securitytests/test_concurrency_postgres.py
.venv/Scripts/bandit -r apps config -ll && .venv/Scripts/pip-audit -r requirements.txt --strict
```

The suite runs in CI with everything else, so a change that reopens any of these holes fails the build.

Evidence: - `apps/securitytests/test_surface_sweep.py` : the route, method, token, junk and role sweep. -
- `apps/securitytests/test_money_auth_idor.py` : sign-in, step-up, money, other people's data, callbacks, leakage. -
- `apps/securitytests/test_concurrency_postgres.py` : the limits race (fails without fix 3, passes with it).